BaseballBrewersAnalysis

August 26, 2025

[1]:
```
import pandas as pd
import numpy as np
import pybaseball as pyball
import matplotlib.pyplot as plt
import requests
import re
import time
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy.interpolate import CubicSpline
from datetime import datetime
from factor_analyzer import FactorAnalyzer
from lifelines import KaplanMeierFitter
from graphviz import Digraph
from graphviz import Source
%matplotlib inline
```

This exploration will analyze the Brewer's game results from the 2020 season to the 2024 season. We intend to analyze which factors could lead to better game performance for the Brewers, as well as how the games impact attendance.

[2]:
```
delay = 5
max_retries = 10

attempt = 0
while attempt < max_retries:
    try:
        brewersFrame = pd.DataFrame(columns = list((pyball.
 ↪schedule_and_record(2000, 'MIL')).columns).append("Year"))
        break
    except (requests.exceptions.RequestException, OSError) as e:
        print(f"Attempt {attempt + 1} failed: {e}")
        attempt += 1
        time.sleep(delay)


for year in range(2000,2025):
    attempt = 0
```

```
    while attempt < max_retries:
        try:
            tempFrame = pyball.schedule_and_record(year, 'MIL')
            tempFrame["Year"] = [year]*len(tempFrame.index)
            break
        except (requests.exceptions.RequestException, OSError) as e:
            print(f"Attempt {attempt + 1} failed: {e}")
            attempt += 1
            time.sleep(delay)
    brewersFrame = pd.concat([brewersFrame, tempFrame])

brewersFrame
```

```
http://www.baseball-reference.com/teams/MIL/2000-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2000-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2001-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2002-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2003-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2004-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2005-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2006-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2007-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2008-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2009-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2010-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2011-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2012-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2013-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2014-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2015-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2016-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2017-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2018-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2019-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2020-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2021-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2022-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2023-schedule-scores.shtml
http://www.baseball-reference.com/teams/MIL/2024-schedule-scores.shtml
```

[2]:

	Date	Tm	Home_Away	Opp	W/L	R	RA	Inn	W-L	Rank	\
1	Monday, Apr 3	MIL	@	CIN	T	3.0	3.0	6.0	0-0	2.0	
2	Tuesday, Apr 4	MIL	@	CIN	W	5.0	1.0	9.0	1-0	1.0	
3	Wednesday, Apr 5	MIL	@	CIN	W	8.0	5.0	9.0	2-0	1.0	
4	Thursday, Apr 6	MIL	@	CIN	L	1.0	5.0	9.0	2-1	2.0	
5	Friday, Apr 7	MIL	@	STL	W	9.0	1.0	9.0	3-1	1.0	
..	…	…	…	…	..	…	…	…	…	…	

```
158  Wednesday, Sep 25  MIL         @  PIT   L  1.0  2.0  9.0  90-68   1.0
159   Thursday, Sep 26  MIL         @  PIT   W  5.0  2.0  9.0  91-68   1.0
160     Friday, Sep 27  MIL      Home  NYM   W  8.0  4.0  9.0  92-68   1.0
161   Saturday, Sep 28  MIL      Home  NYM   W  6.0  0.0  9.0  93-68   1.0
162     Sunday, Sep 29  MIL      Home  NYM   L  0.0  5.0  9.0  93-69   1.0

     …       Win        Loss      Save  Time D/N Attendance   cLI Streak  \
1    …      None        None      None  1:51   D    55596.0  1.01    NaN
2    …    Bruske  Williamson      None  3:11   N    16761.0  1.04    1.0
3    …    Haynes      Parris   Wickman  3:08   N    23059.0  1.08    2.0
4    …   Villone     Navarro      None  2:53   N    20909.0  1.14   -1.0
5    …      Bere       Benes      None  3:25   N    33089.0  1.10    1.0
..   …       …          …       …    …  ..        …    …     …
158  …     Ortiz     Peralta   Chapman  2:11   N    15965.0   .51   -1.0
159  …    Civale      Keller  Williams  2:52   D    16797.0   .03    1.0
160  …      Ross      Manaea    Megill  3:05   N    33996.0   .05    2.0
161  …     Myers    Quintana      None  2:31   N    39637.0   .08    3.0
162  …  Peterson         Rea      None  2:34   D    33754.0   .05   -1.0

     Orig. Scheduled  Year
1                None  2000
2                None  2000
3                None  2000
4                None  2000
5                None  2000
..                …    …
158              None  2024
159              None  2024
160              None  2024
161              None  2024
162              None  2024

[3949 rows x 21 columns]
```

Here, we are using the pybaseball library to get information from the Brewer's performance from the 2020 season to the 2024 season. We will decompose the date column using regex to sepeare the year from the month and day.

[3]:
```
newFrame = brewersFrame.copy(deep=True)
newFrame = newFrame.reset_index(drop=True)
newFrame["Date"] = newFrame["Date"].str.extract(r",\s*(.*)")
newFrame["Date"] = newFrame["Date"].str.replace(r"\s*\([^)]*\)", "", regex=True)
newFrame
```

[3]:
```
         Date   Tm Home_Away  Opp W/L    R   RA  Inn    W-L  Rank  …  \
0      Apr 3  MIL         @  CIN   T  3.0  3.0  6.0    0-0   2.0  …
1      Apr 4  MIL         @  CIN   W  5.0  1.0  9.0    1-0   1.0  …
```

```
2      Apr 5  MIL         @  CIN   W  8.0  5.0  9.0    2-0   1.0  …
3      Apr 6  MIL         @  CIN   L  1.0  5.0  9.0    2-1   2.0  …
4      Apr 7  MIL         @  STL   W  9.0  1.0  9.0    3-1   1.0  …
…      …  …       …   …  ..  …  …  …    …    …  …
3944  Sep 25  MIL         @  PIT   L  1.0  2.0  9.0  90-68   1.0  …
3945  Sep 26  MIL         @  PIT   W  5.0  2.0  9.0  91-68   1.0  …
3946  Sep 27  MIL      Home  NYM   W  8.0  4.0  9.0  92-68   1.0  …
3947  Sep 28  MIL      Home  NYM   W  6.0  0.0  9.0  93-68   1.0  …
3948  Sep 29  MIL      Home  NYM   L  0.0  5.0  9.0  93-69   1.0  …

           Win        Loss      Save  Time D/N Attendance   cLI Streak  \
0         None        None      None  1:51   D    55596.0  1.01    NaN
1       Bruske  Williamson      None  3:11   N    16761.0  1.04    1.0
2       Haynes      Parris   Wickman  3:08   N    23059.0  1.08    2.0
3      Villone     Navarro      None  2:53   N    20909.0  1.14   -1.0
4         Bere       Benes      None  3:25   N    33089.0  1.10    1.0
…          …          …       …   …  ..       …   …    …
3944     Ortiz     Peralta   Chapman  2:11   N    15965.0   .51   -1.0
3945    Civale      Keller  Williams  2:52   D    16797.0   .03    1.0
3946      Ross      Manaea    Megill  3:05   N    33996.0   .05    2.0
3947     Myers    Quintana      None  2:31   N    39637.0   .08    3.0
3948  Peterson         Rea      None  2:34   D    33754.0   .05   -1.0

      Orig. Scheduled  Year
0                None  2000
1                None  2000
2                None  2000
3                None  2000
4                None  2000
…                …   …
3944             None  2024
3945             None  2024
3946             None  2024
3947             None  2024
3948             None  2024

[3949 rows x 21 columns]
```

Afterwards, we want to convert the win and loss categorical variable into discrete numbers, where 1 is a win, 0 is a tie, and -1 is a loss.

[4]:
```python
def Win_Loss_Conversion(char):
    if char == "W":
        return 1
    elif char == "L":
        return -1
    else:
```

```
        return 0


newFrame["WL_Numeric"] = newFrame["W/L"].apply(Win_Loss_Conversion)
```

To do further analysis on game attendance, we now want to drop all na values in the attendance column.

[5]:
```
attendance_schedule = newFrame["Attendance"]
attendance_schedule=attendance_schedule.dropna()
attendance_schedule

filtered = newFrame[newFrame.index.isin(attendance_schedule.index)]
filtered
```

[5]:

	Date	Tm	Home_Away	Opp	W/L	R	RA	Inn	W-L	Rank	…	\
0	Apr 3	MIL	@	CIN	T	3.0	3.0	6.0	0-0	2.0	…	
1	Apr 4	MIL	@	CIN	W	5.0	1.0	9.0	1-0	1.0	…	
2	Apr 5	MIL	@	CIN	W	8.0	5.0	9.0	2-0	1.0	…	
3	Apr 6	MIL	@	CIN	L	1.0	5.0	9.0	2-1	2.0	…	
4	Apr 7	MIL	@	STL	W	9.0	1.0	9.0	3-1	1.0	…	
…	…	…	…	…	..	…	…	…	…	…	…	
3944	Sep 25	MIL	@	PIT	L	1.0	2.0	9.0	90-68	1.0	…	
3945	Sep 26	MIL	@	PIT	W	5.0	2.0	9.0	91-68	1.0	…	
3946	Sep 27	MIL	Home	NYM	W	8.0	4.0	9.0	92-68	1.0	…	
3947	Sep 28	MIL	Home	NYM	W	6.0	0.0	9.0	93-68	1.0	…	
3948	Sep 29	MIL	Home	NYM	L	0.0	5.0	9.0	93-69	1.0	…	

	Loss	Save	Time	D/N	Attendance	cLI	Streak	Orig. Scheduled	\
0	None	None	1:51	D	55596.0	1.01	NaN	None	
1	Williamson	None	3:11	N	16761.0	1.04	1.0	None	
2	Parris	Wickman	3:08	N	23059.0	1.08	2.0	None	
3	Navarro	None	2:53	N	20909.0	1.14	-1.0	None	
4	Benes	None	3:25	N	33089.0	1.10	1.0	None	
…	…	…	…	..	…	…	…	…	
3944	Peralta	Chapman	2:11	N	15965.0	.51	-1.0	None	
3945	Keller	Williams	2:52	D	16797.0	.03	1.0	None	
3946	Manaea	Megill	3:05	N	33996.0	.05	2.0	None	
3947	Quintana	None	2:31	N	39637.0	.08	3.0	None	
3948	Rea	None	2:34	D	33754.0	.05	-1.0	None	

	Year	WL_Numeric
0	2000	0
1	2000	1
2	2000	1
3	2000	-1
4	2000	1

```
…     …         …
3944  2024         -1
3945  2024          1
3946  2024          1
3947  2024          1
3948  2024         -1

[3875 rows x 22 columns]
```

Now, for every month and day for each game, we want to average the number of attendance that arrived across all seasons. In the table bellow, the year 1900 is irrelevant.

[6]:
```
filtered["Date"] = pd.to_datetime(filtered["Date"], format="%b %d")
agg = filtered.groupby('Date')['Attendance'].mean().reset_index()
agg['DaysSinceStart'] = (agg['Date'] - agg['Date'].min()).dt.days
agg
```

```
/var/folders/y2/kgz80nr17fx6lx0f9pnpm9xw0000gn/T/ipykernel_97370/4175552531.py:1
: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  filtered["Date"] = pd.to_datetime(filtered["Date"], format="%b %d")
```

[6]:

	Date	Attendance	DaysSinceStart
0	1900-03-28	45304.000000	0
1	1900-03-29	38981.000000	1
2	1900-03-30	33629.500000	2
3	1900-03-31	38729.857143	3
4	1900-04-01	23637.200000	4
..	…	…	…
189	1900-10-03	35760.000000	189
190	1900-10-04	31209.000000	190
191	1900-10-05	21732.500000	191
192	1900-10-06	40038.000000	192
193	1900-10-07	31761.000000	193

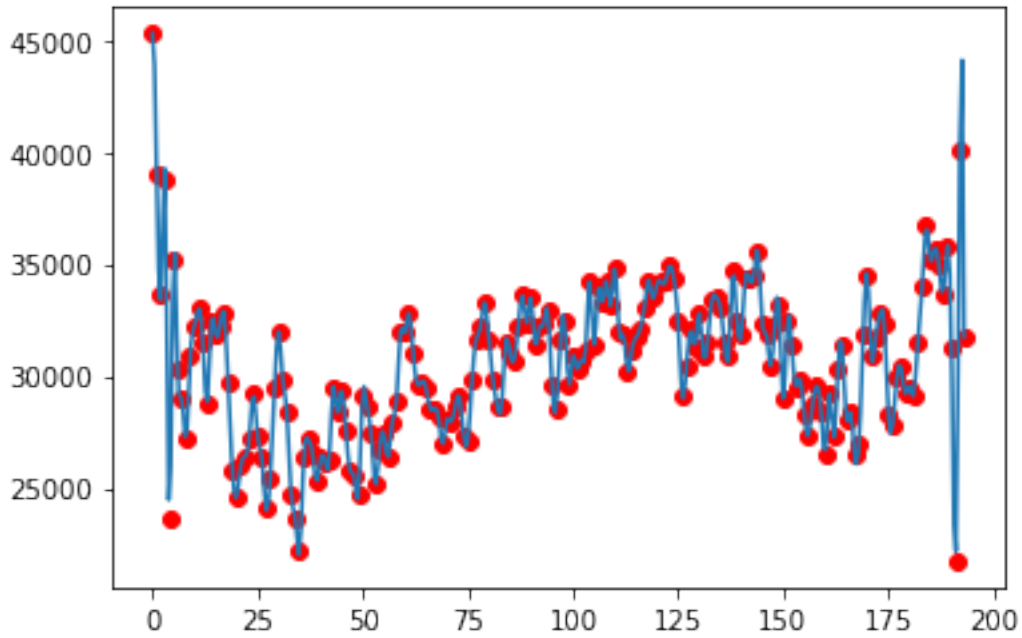
[194 rows x 3 columns]

To best see the trend in the data, we can create a cubic spline to see the fluctuation between the dates and attendance.

[7]:
```
cs = CubicSpline(agg['DaysSinceStart'], agg["Attendance"])
date_mesh = np.linspace(agg['DaysSinceStart'].min(), agg['DaysSinceStart'].
 ↪max(), 400)
att_spline = cs(date_mesh)
```

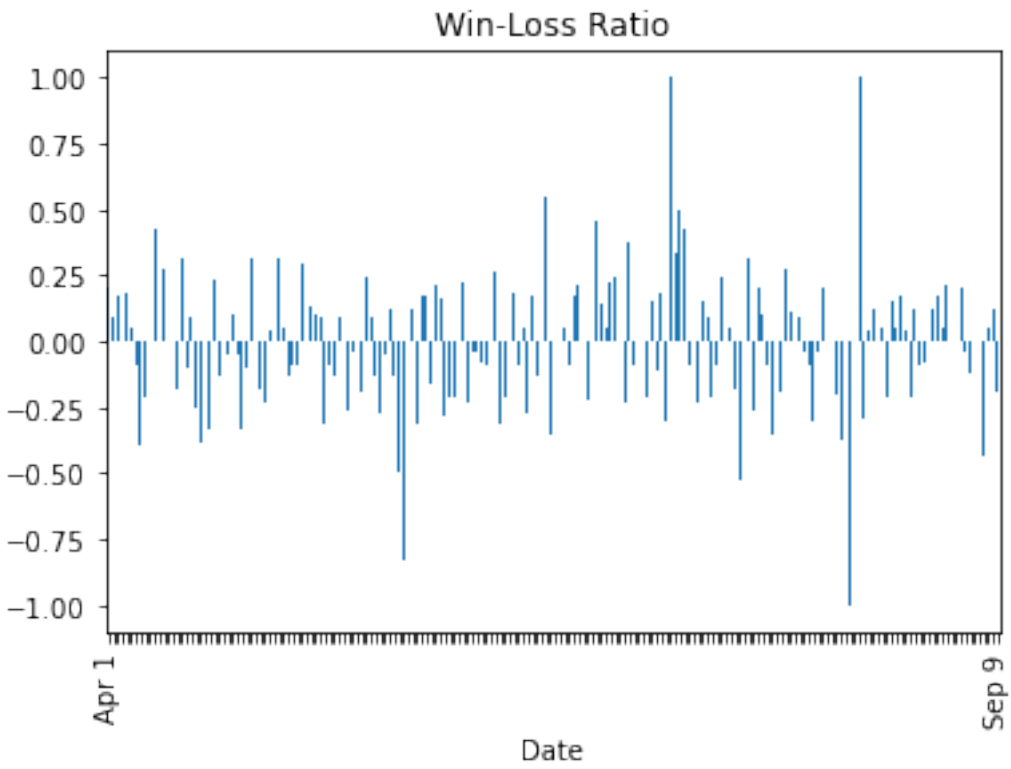
```
plt.scatter(agg['DaysSinceStart'], agg['Attendance'], color='red', label='Data␣
 ↪points')
plt.plot(date_mesh, att_spline, label='Cubic spline interpolation')
```

[7]: [<matplotlib.lines.Line2D at 0x7fe22c18a400>]



Moreover, we can plot the average win-loss ratio of a certain day to see if there are any trends.

[8]:
```
winloss = newFrame.groupby("Date")["WL_Numeric"].mean()

ax= winloss.plot(kind = "bar")

tick_positions = range(len(winloss))
tick_labels = winloss.index

custom_labels = [''] * len(tick_labels)
custom_labels[0] = tick_labels[0]
custom_labels[-1] = tick_labels[-1]

ax.set_xticks(tick_positions)
ax.set_xticklabels(custom_labels)

plt.title("Win-Loss Ratio")
plt.show()
```



We then can see whether opponent and homeaway has any impact on the wins and losses of the Brewers. Here, we are using an OLS model to acount for the ternary outcomes.

[9]:
```
model1 = smf.ols('WL_Numeric ~ C(Opp) * C(Home_Away)', data=newFrame).fit()
anova1 = sm.stats.anova_lm(model1, typ=2)
print(anova1)
```

```
                       sum_sq      df         F    PR(>F)
C(Opp)                 33.891997    32.0  1.158106  0.250640
C(Home_Away)            0.170459     1.0  0.186390  0.665963
C(Opp):C(Home_Away)    28.504274    32.0  0.974005  0.507525
Residual             3552.042254  3884.0       NaN       NaN
```

```
/Users/elliotbarta/opt/anaconda3/lib/python3.9/site-
packages/statsmodels/base/model.py:1871: ValueWarning: covariance of constraints
does not have full rank. The number of constraints is 32, but rank is 31
  warnings.warn('covariance of constraints does not have full '
/Users/elliotbarta/opt/anaconda3/lib/python3.9/site-
packages/statsmodels/base/model.py:1871: ValueWarning: covariance of constraints
does not have full rank. The number of constraints is 32, but rank is 31
  warnings.warn('covariance of constraints does not have full '
```

From the ANOVA, it is clear that neither Home_Away, Opponent, or its interaction are statistically significant at a 5% level. The results may be limited given the use on OLS with discrete variables

as response variable.

To continue the exploration, we want to apply Survival Analysis to see the probability to the Brewer's game streak continuing.

[10]:
```
filtered

streaks = newFrame["Streak"]
streaks=streaks.dropna()
streaks

filtered = filtered[filtered.index.isin(streaks.index)]
filtered
```

[10]:

	Date	Tm	Home_Away	Opp	W/L	R	RA	Inn	W-L	Rank	…	\
1	1900-04-04	MIL	@	CIN	W	5.0	1.0	9.0	1-0	1.0	…	
2	1900-04-05	MIL	@	CIN	W	8.0	5.0	9.0	2-0	1.0	…	
3	1900-04-06	MIL	@	CIN	L	1.0	5.0	9.0	2-1	2.0	…	
4	1900-04-07	MIL	@	STL	W	9.0	1.0	9.0	3-1	1.0	…	
5	1900-04-08	MIL	@	STL	L	8.0	10.0	9.0	3-2	2.0	…	
…	…	…	…	…	..	…	…	…	…	…	…	
3944	1900-09-25	MIL	@	PIT	L	1.0	2.0	9.0	90-68	1.0	…	
3945	1900-09-26	MIL	@	PIT	W	5.0	2.0	9.0	91-68	1.0	…	
3946	1900-09-27	MIL	Home	NYM	W	8.0	4.0	9.0	92-68	1.0	…	
3947	1900-09-28	MIL	Home	NYM	W	6.0	0.0	9.0	93-68	1.0	…	
3948	1900-09-29	MIL	Home	NYM	L	0.0	5.0	9.0	93-69	1.0	…	

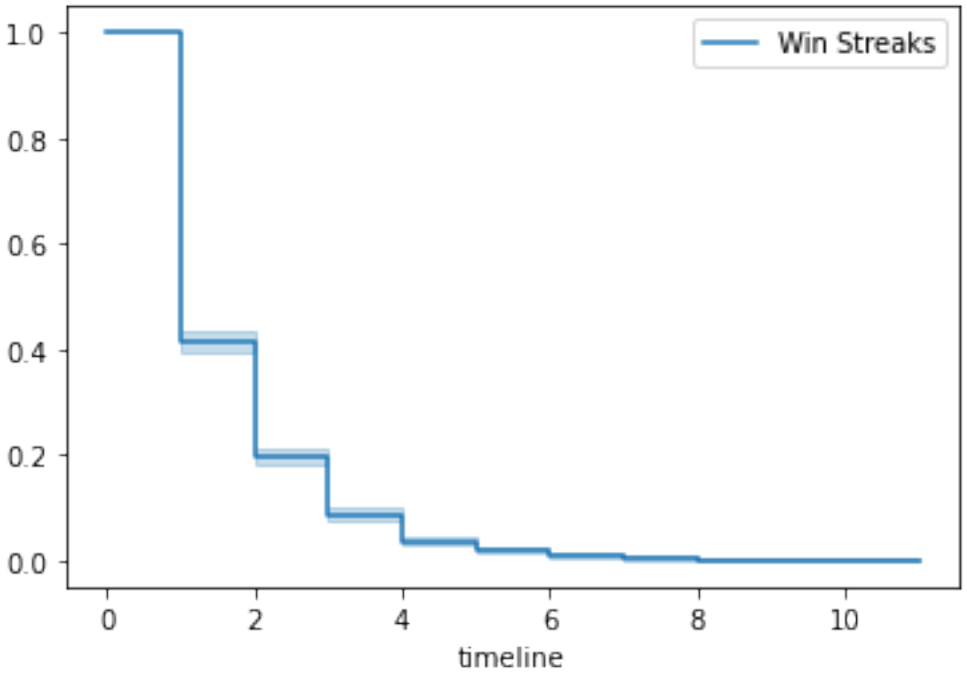
	Loss	Save	Time	D/N	Attendance	cLI	Streak	Orig. Scheduled	\
1	Williamson	None	3:11	N	16761.0	1.04	1.0	None	
2	Parris	Wickman	3:08	N	23059.0	1.08	2.0	None	
3	Navarro	None	2:53	N	20909.0	1.14	-1.0	None	
4	Benes	None	3:25	N	33089.0	1.10	1.0	None	
5	Woodard	Veres	3:50	D	40909.0	1.14	-1.0	None	
…	…	…	…	..	…	…	…	…	
3944	Peralta	Chapman	2:11	N	15965.0	.51	-1.0	None	
3945	Keller	Williams	2:52	D	16797.0	.03	1.0	None	
3946	Manaea	Megill	3:05	N	33996.0	.05	2.0	None	
3947	Quintana	None	2:31	N	39637.0	.08	3.0	None	
3948	Rea	None	2:34	D	33754.0	.05	-1.0	None	

	Year	WL_Numeric
1	2000	1
2	2000	1
3	2000	-1
4	2000	1
5	2000	-1
…	…	…
3944	2024	-1

```
3945  2024          1
3946  2024          1
3947  2024          1
3948  2024         -1

[3874 rows x 22 columns]
```

[11]:
```
# Survival Analysis

streak_id = (filtered['WL_Numeric'] != filtered['WL_Numeric'].shift()).cumsum()
streaks = filtered['WL_Numeric'].groupby(streak_id).agg(['first','size'])
streaks.columns = ['result','length']

kmf = KaplanMeierFitter()

kmf.fit(streaks["length"], event_observed=[1]*len(streaks), label="Win Streaks")
kmf.plot_survival_function()
```

[11]: <AxesSubplot:xlabel='timeline'>



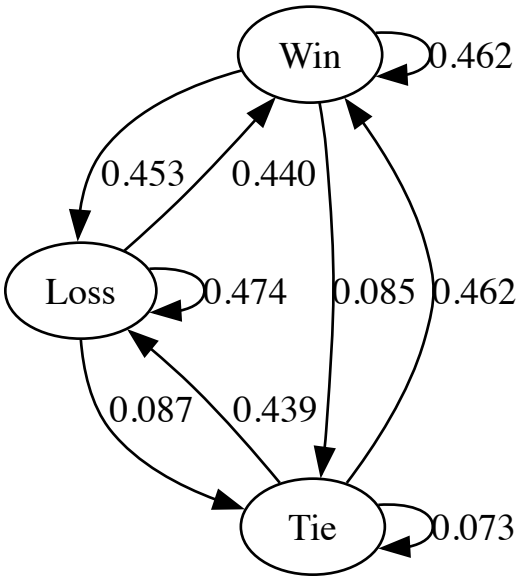
From the chart, we can see that the probability of the streak continuing dimnishes rapidly. To further contextualize the survival analysis, we can make a Markov Chain that illustrates the prior probabilities of winning.

[12]:
```
results = filtered["WL_Numeric"]

# Shifted series = previous game
prev = results.shift().dropna().astype(int)
curr = results[1:].astype(int)

# Build transition table
transitions = pd.crosstab(prev, curr, normalize=0)
transitions.index = ["Prev Loss","Prev Tie","Prev Win"]
transitions.columns = ["Next Loss","Next Tie","Next Win"]
print(transitions)
```

```
           Next Loss  Next Tie  Next Win
Prev Loss   0.473684  0.086786  0.439530
Prev Tie    0.440729  0.072948  0.486322
Prev Win    0.452787  0.085324  0.461889
```

[13]:
```
dot = Digraph(engine="circo")

dot.engine = 'neato'
dot.node("Win")
dot.node("Loss")
dot.node("Tie")

dot.edge("Win", "Win", label="0.462")
dot.edge("Win", "Loss", label="0.453")
dot.edge("Win", "Tie", label = "0.085")
dot.edge("Loss", "Win", label=" 0.440")
dot.edge("Loss", "Loss", label="0.474")
dot.edge("Loss", "Tie", label="0.087")
dot.edge("Tie", "Win", label="0.462")
dot.edge("Tie", "Loss", label="0.439")
dot.edge("Tie", "Tie", label="0.073")

Source(dot.source)
```

[13]:



From the graph, it is difficult to make any concrete conclusions on whether the Brewer's experiens a "hot-hands" effect in either winning, loosing, or getting a tie. Nonetheless, it is interesting how after getting a tie, the Brewer's are more likely to win than to lose.